Aplicación — Guía 6

RCCD 2C 2026 · Resumen de la teórica 16 (Nivel de Aplicación), las prácticas de DNS y de Correo/HTTP, y los temas de la guía 6.

0 · La idea en criollo (ELI5)

Las capas de abajo ya saben llevar bytes de una compu a otra (IP) y entregarlos sin perderse a un **proceso** (TCP/UDP). La capa de aplicación es el **idioma** que hablan dos programas por encima de eso: qué mensajes existen, qué campos tienen y quién habla primero. Casi todo es **pedido → respuesta, en texto ASCII legible**.

- **DNS** es la **guía telefónica** de Internet: vos sabés el nombre (`www.dc.uba.ar`) y necesitás el número (la IP). Nadie tiene la guía entera. Cada zona tiene su pedacito, y hay un "secretario" (el **resolver**) que va preguntando hasta encontrarla y se anota la respuesta por un rato (**caché + TTL**).
- **El mail** es el **correo postal**. Tu programa de mail le deja la carta a tu oficina de correo (**servidor saliente, SMTP**). Esa oficina busca en DNS cuál es la oficina del destinatario (**registro MX**) y se la lleva (**SMTP** otra vez). El destinatario la retira de su casilla con **POP3** (se la lleva a casa) o con **IMAP** (la lee en la oficina, sincronizada).
- **La web (HTTP)** es pedir cosas en un mostrador: "dame `/index.html`" y te la dan. La página trae referencias a otras cosas (imágenes, CSS), y el navegador las pide **una por una**. El servidor **no se acuerda de vos** entre pedido y pedido (stateless): para eso existen las **cookies**.

> **El orden de las cosas siempre es:** nombre → **DNS** (UDP 53) → IP → **conexión TCP** (handshake: 1 RTT) → mensajes de aplicación (HTTP/SMTP/POP3/IMAP) → cierre. Los ejercicios integradores (10, 11, 12) consisten en escribir esa cadena completa para cada host que aparece.

1 · Generalidades de la capa de aplicación

- **Proceso**: programa corriendo en un host. Dos procesos en hosts distintos se hablan con un protocolo de aplicación. Un **socket** es la "puerta" del proceso hacia el transporte.
- A un proceso se lo identifica con **IP + puerto**. Con la IP sola no alcanza, porque en un host corren muchos procesos.
- **Agente de usuario**: la interfaz entre el usuario y la red (navegador, Thunderbird). Ojo: **aplicación ≠ protocolo** (Firefox ≠ HTTP).
- **Cliente-servidor**: el cliente **habla primero** y pide. El servidor atiende en un puerto conocido. También existen P2P, modelos híbridos y RPC.
- Un protocolo define: los **tipos de mensaje** (pedido/respuesta), su **sintaxis** (campos), su **semántica** (qué significan) y las **reglas** de cuándo se envían. Los públicos están en RFCs (interoperabilidad).

TCP vs UDP — qué elige cada app

TCP: orientado a conexión, confiable, control de flujo y de congestión. **UDP**: nada de eso, solo multiplexa con puertos. Ninguno de los dos garantiza tiempos ni ancho de banda. Las apps se eligen según tres criterios: **tolerancia a pérdidas**, **sensibilidad al delay** y **ancho de banda** (por ejemplo, el mail no tolera pérdidas pero sí demora; la voz tolera pérdidas pero no demora).

Protocolo	Transporte	Puerto	Para qué
DNS	**UDP** (TCP para transferencias de zona / respuestas grandes)	53	nombre → IP y otros registros
HTTP / HTTPS	TCP	80 / 443	web
SMTP	TCP	25 (587 submission)	enviar mail: UA → saliente → entrante
POP3	TCP	110 (995 con TLS)	bajar mail del buzón
IMAP	TCP	143 (993 con TLS)	leer mail en el servidor, sincronizado

Ej. 1 de la guía (5 clientes a la vez): alcanza **1 servidor**, **1 IP** y **1 puerto** (el 80). TCP distingue cada conexión por la 4-upla (`IP orig, puerto orig, IP dest, puerto dest`): los clientes usan puertos efímeros distintos, así que no chocan. Si crece la cantidad de clientes, el límite es de **capacidad** del servidor, no de puertos. Se escala con más servidores detrás de un balanceador, o con DNS que devuelva varias IPs.

2 · DNS — Domain Name System (RFC 1034/1035)

Por qué existe y cómo está organizado

Antes había un `hosts.txt` central: no escalaba y era propenso a errores. DNS es una **base de datos distribuida y jerárquica** que separa el **nombre** de un host de su **dirección**.

- **Namespace jerárquico**: raíz `.` → TLD (`ar`, `com`; ccTLD = de país) → `uba` → `dc`. Dos hermanos no pueden llamarse igual. Lo administra ICANN.
- **FQDN** (nombre absoluto, termina en punto: `mail.dc.uba.ar.`) vs **PQDN** (relativo: `mail`). En un archivo de zona, **un nombre sin punto final se completa con la zona**: `mailserver` en la zona uba.ar es `mailserver.uba.ar`, y en la zona dc.uba.ar es `mailserver.dc.uba.ar` (son distintos).
- **Zona** = porción del árbol bajo una misma autoridad administrativa. Una zona puede **delegar** un subdominio a otra zona: uba.ar delega dc.uba.ar con un registro NS.

Componentes

- **Name servers autoritativos**: tienen los registros oficiales de su zona. Hay un **primario** (master, el que se edita) y **secundarios** (copias que se actualizan desde el primario cuando cambia el *serial* del SOA).
- **Resolver** (servidor **no autoritativo**, local o del ISP): recibe las consultas de los clientes, sale a resolverlas y **cachea** las respuestas.
- En el host, la app llama al **resolver de biblioteca** (`gethostbyname`), que manda un paquete **UDP** al resolver local. Recién con la IP la app abre la conexión TCP.

Registros (Resource Records): `nombre TTL clase tipo valor`

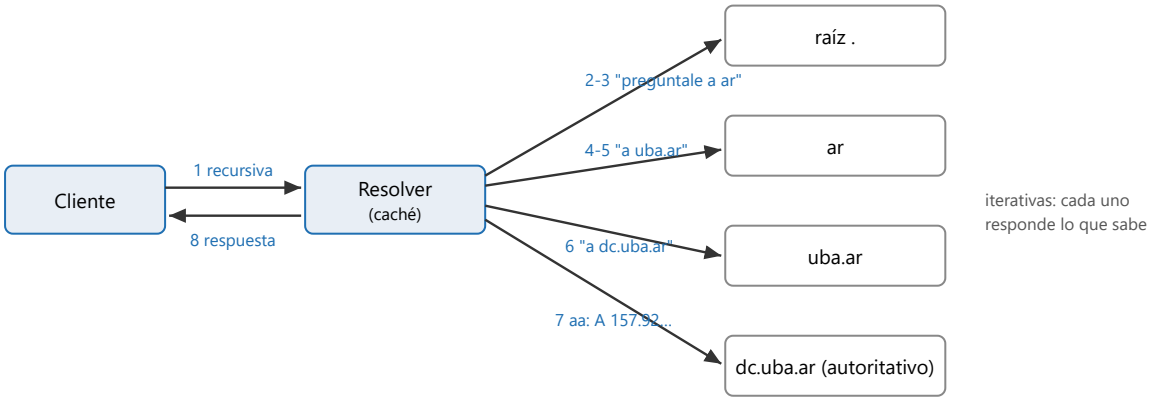
Tipo	Significa	Valor
A / AAAA	dirección de un host	IPv4 / IPv6
CNAME	alias: "este nombre es otro nombre"	nombre canónico (se sigue la cadena hasta un A)
MX	servidor que **recibe** el mail del dominio	prioridad + nombre (**menor número = mayor prioridad**)
NS	name server autoritativo de la zona	nombre del servidor
SOA	comienzo de autoridad (uno por zona)	primario, mail admin, serial, refresh, retry, expire, minimum
PTR	resolución inversa (IP → nombre)	nombre
TXT / HINFO	texto libre / descripción del host	ASCII

SOA en detalle (`dc.uba.ar. IN SOA ns1.dc.uba.ar. admin.dc.uba.ar. (serial refresh retry expire minimum)`):

- El **primer nombre es el primario**. El segundo es el mail del admin, con la @ reemplazada por un punto.
- **serial**: versión de la zona (fecha AAAAMMDDnn). Si no se incrementa, los secundarios no copian los cambios.
- **refresh**: cada cuánto el secundario consulta al primario. **retry**: cuánto espera para reintentar si falló. **expire**: cuándo deja de responder si el primario no contesta.
- **minimum**: TTL de la **caché negativa**, o sea cuánto se recuerda un "ese nombre no existe".

Consultas y respuestas

	Qué es	Quién la usa
Consulta recursiva	"resolvémelo vos y traeme la respuesta final"	cliente → resolver
Consulta iterativa	"decime lo que sepas": la respuesta o **a quién preguntarle** (referral)	resolver → root, TLD, autoritativos
Respuesta autoritativa (flag aa)	viene de un servidor primario o secundario de la zona	autoritativo
Respuesta no autoritativa	viene de la **caché** de alguien	resolver



- **Referral**: cuando un servidor no es autoritativo del nombre pedido (porque la zona está delegada), responde con el **NS de la zona hija** en la sección AUTHORITY.
- **Glue record**: si el NS de la zona hija está *dentro* de la zona hija (por ejemplo, `servidores.dc.uba.ar` es NS de `dc.uba.ar`), la zona padre incluye también su registro **A** (en ADDITIONAL). Sin ese registro habría una

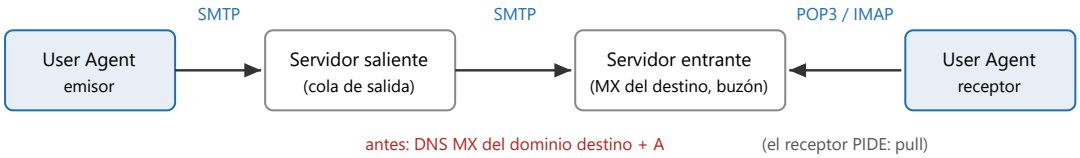
dependencia circular.

- **Secciones de una respuesta**: ANSWER (lo pedido), AUTHORITY (los NS), ADDITIONAL (registros A de regalo, por ejemplo el A del servidor MX, que ahorran otra consulta).
- **Caché**: el resolver guarda **todo** lo que recibe (answer, NS, glue) durante el **TTL** de cada registro. La próxima consulta arranca desde el **ancestro más cercano que conozca**, no desde la raíz.

> **TTL y cambios de IP** (salió en el 2do parcial de 2024): si un registro A con TTL de 3 días cambia de IP, los resolvers que lo tenían cacheado siguen respondiendo la **IP vieja** (como respuesta no autoritativa) hasta que vence el TTL. Recién después vuelven a preguntar al autoritativo y obtienen la nueva. El cambio **no se propaga**: hay que esperar que expiren las cachés. Por eso se baja el TTL antes de una migración.

CDN vía DNS: el autoritativo de la CDN devuelve una IP distinta según dónde esté el que pregunta. Así cada usuario termina en la réplica más cercana.

3 · Correo electrónico — SMTP, formato, MIME, POP3, IMAP



- **SMTP es push**: siempre lo inicia el que *tiene* el mail. Por eso para **leer** hace falta otro protocolo (POP3/IMAP), que es **pull**.
- El servidor saliente busca el **MX** del dominio del destinatario (y su A) en DNS, y se conecta por SMTP al puerto 25. Si hay varios MX, prueba por prioridad. Puede haber **relays** intermedios.
- **Webmail**: entre el navegador y el servidor de webmail se habla **HTTP(S)**, no SMTP ni POP3. Es el servidor de webmail el que después usa SMTP hacia afuera y lee el buzón por su cuenta.

Sesión SMTP (comando ASCII → código numérico + frase)

```
S: 220 mail.destino.com ESMTP listo
C: HELO origen.com          (o EHLO)
S: 250 Hello
C: MAIL FROM: <yo@origen.com>
S: 250 Ok
C: RCPT TO: <vos@destino.com>   (se repite)
S: 250 Ok
C: DATA
S: 354 End with <CRLF>.<CRLF>
C: From: ... To: ... Subject: ...
C: (línea en blanco) cuerpo...
C: .
S: 250 Ok: queued
C: QUIT
S: 221 Bye
```

Códigos: 2xx ok (220 listo, 250 ok, 221 chau) · 3xx esperando (354 mandá los datos) · 4xx error temporal (421) · 5xx error permanente (550 no existe el buzón).

Otros comandos: RSET (cancela la transacción), VRFY.

Sobre vs. carta: `MAIL FROM / RCPT TO` son el **sobre** (envelope), que es lo que usa SMTP para enrutar. `From: / To:` van dentro de DATA: son la **carta** (headers RFC 822), lo que ve el usuario. Pueden no coincidir (BCC, listas de correo, spam).

Las tres fases son saludo, transferencia y cierre. Por **una** conexión TCP se pueden mandar varios mails.

Formato del mensaje (RFC 822) y MIME (RFC 2045)

Un mensaje es: **headers** (`From:`, `To:`, `Subject:`, `Date:`…), una **línea en blanco** y el **cuerpo**. Originalmente era **solo ASCII de 7 bits**.

MIME resuelve cómo mandar imágenes, HTML, adjuntos y acentos **sin cambiar SMTP ni los servidores** (ej. 8 de la guía). Solo agrega headers y una convención de codificación:

```
MIME-Version: 1.0
Content-Type: image/jpeg                ← tipo/subtipo (text/html, application/pdf,
video/mpeg...)
Content-Transfer-Encoding: base64       ← cómo se pasó a ASCII
(línea en blanco)
/9j/4AAQSkZJRgABAQEASABIAAD...          ← datos codificados
```

- Hay 7 tipos de primer nivel: text, image, audio, video, application, multipart, message.
- **multipart/mixed; boundary="XYZ"**: varias partes en un solo mensaje, separadas por `--XYZ`. Cada parte tiene su propio Content-Type (framing con delimitadores).
- **base64**: 3 bytes binarios → 4 caracteres ASCII (+33% de tamaño). **quoted-printable**: para texto casi-ASCII (acentos).
- HTTP **reusa** los tipos MIME en su header `Content-Type` (ej. 9 de la guía).

POP3 vs IMAP

	POP3 (RFC 1939, TCP 110)	IMAP (RFC 3501, TCP 143)
Paradigma	**download and delete**: bajo y borro del servidor	**keep and sync**: todo queda en el servidor
Estado	no sincroniza (leído/no leído, carpetas)	carpetas, flags (\Seen, \Deleted...), búsquedas en el servidor, varios dispositivos
Descarga parcial	no (mensaje entero)	sí: solo headers o una parte MIME (FETCH)
Fases / comandos	autorización (USER, PASS) → transacción (STAT, LIST, RETR n, DELE n, RSET) → **update** al QUIT (ahí se borra)	LOGIN, LIST, SELECT, FETCH, STORE, EXPUNGE, LOGOUT. Comandos con **tag** (a1, a2...) → se pueden mandar varios a la vez
Respuestas	`+OK` / `-ERR`	`a1 OK` / `NO` (falla) / `BAD` (sintaxis); `*` = respuesta no etiquetada (datos)

4 · Web — HTTP

- Una **página** es un HTML base más **objetos** referenciados (imágenes, CSS, JS). Cada objeto tiene su **URL** = host + ruta.
- HTTP es **cliente/servidor sobre TCP (80)**, en **texto ASCII**, y es **sin estado**: el servidor no recuerda pedidos anteriores. Es más simple y robusto ante caídas.

```
GET /dir/pagina.html HTTP/1.1      ← línea de
pedido
Host: www.dc.uba.ar                ←
obligatorio en 1.1
User-Agent: Mozilla/5.0
Accept-Language: es
Connection: close
(línea en blanco)                  ← fin de
headers
[cuerpo: solo POST/PUT]
```

```
HTTP/1.1 200 OK                    ← línea de
estado
Date: Thu, 06 Aug 1998 12:00:15 GMT
Server: Apache/1.3.0
Last-Modified: Mon, 22 Jun 1998 ...
Content-Length: 6821
Content-Type: text/html
(línea en blanco)
<html>...                          ← cuerpo
```

Métodos

Método	Qué hace	Detalle
GET	pide un recurso	sin cuerpo; los parámetros van en la URL (`?a=1&b=2`); idempotente y cacheable
HEAD	igual que GET pero **sin cuerpo** en la respuesta	para ver si existe, el tipo, el tamaño o la fecha de modificación (validar caché)
POST	envía datos (formularios, crear un recurso)	datos en el **cuerpo**, con `Content-Type` y `Content-Length`; **no idempotente**
PUT / DELETE	sube/reemplaza un recurso / lo borra	en HTTP/1.1

GET vs POST (salió en el 2do parcial de 2024): GET *pide*, no tiene cuerpo, sus parámetros quedan visibles en la URL (y en el historial y los logs), se puede cachear, y repetirlo no cambia nada. POST *manda* datos en el cuerpo, sirve para cambiar estado en el servidor (crear, comprar) y repetirlo puede duplicar la acción.

Códigos de estado

1xx informativo · **2xx** éxito (200 OK, 201 Created) · **3xx** redirección (301 Moved Permanently + `Location:`, **304 Not Modified**) · **4xx** error del cliente (400 Bad Request, 401 Unauthorized, 404 Not Found) · **5xx** error del servidor (500, 505 HTTP Version Not Supported).

Headers clave

Header	Dónde	Para qué
Host	pedido	dominio pedido. Permite **virtual hosting** (varios sitios en una misma IP)
Content-Type / Content-Length	ambos	tipo MIME y tamaño del cuerpo
If-Modified-Since	pedido	**GET condicional**: si no cambió → **304** sin cuerpo; si cambió → 200 con el archivo
Last-Modified / Cache-Control / Expires	respuesta	info para cachear
Authorization / WWW-Authenticate	pedido / respuesta 401	credenciales (van en **cada** pedido, porque es stateless)
Set-Cookie / Cookie	respuesta / pedido	estado sobre un protocolo stateless
Connection: close / keep-alive	ambos	cerrar o mantener la conexión TCP
Via	pedido (lo agrega el proxy)	marca que pasó por un proxy

Tiempo de respuesta en RTTs

Para cada objeto se suman: **1 RTT** de handshake TCP (SYN → SYN+ACK; el ACK viaja junto con el GET) + **1 RTT** de GET → respuesta + tiempo de transmisión. El cierre (FIN) **no suma**, porque no bloquea el pedido siguiente.

Modo	Cómo	Página con HTML + N objetos
HTTP/1.0 no persistente	una conexión TCP **por objeto**	2 RTT × (1 + N)
1.1 persistente sin pipelining	misma conexión; el próximo pedido sale cuando llegó la respuesta anterior	2 RTT (HTML) + N × 1 RTT
1.1 persistente con pipelining	todos los pedidos en ráfaga, sin esperar	2 RTT (HTML) + 1 RTT

Ojo: **no se pueden pedir los objetos hasta tener el HTML**, porque ahí es donde aparecen. Tampoco suman: los navegadores pueden abrir **conexiones paralelas** (en 1.0 los N objetos irían en paralelo: 2 + 2 RTT). Si no se dice nada, se asume secuencial.

Qué tags de HTML disparan pedidos automáticos: `<img src>`, `<link rel="stylesheet" href>`, `<script src>`, `<audio/video src>`, `<input type="image" src>`, `<iframe src>` (que es otra página entera, con sus propios objetos). **No disparan**: `<a href>` (requiere clic) ni `<form action>` (requiere submit). Si un objeto está en **otro host**, requiere otra consulta DNS y otra conexión TCP.

Proxy / caché web, cookies, CDN

- **Proxy**: el navegador le pide **todo** al proxy (necesita resolver por DNS solo la IP del proxy, si no la tiene configurada). El proxy actúa de servidor para el cliente y de cliente para el servidor. **Hit**: responde desde su caché. **Miss**: le pide al servidor de origen (con DNS y TCP propios), cachea y reenvía. Ventajas: menos tiempo de respuesta y menos tráfico WAN. Puede validar con GET condicional.
- **Cookies**: en la primera respuesta el servidor manda `Set-Cookie: 1678` y lo guarda en su base de datos. El navegador lo devuelve en cada pedido con `Cookie: 1678`. Tiene 4 partes: header de respuesta, header de pedido, archivo en el cliente y base de datos en el servidor. Sirve para sesiones y carritos (y para **tracking**).
- **CDN**: servidores de réplica dentro de los ISPs. El usuario llega a la réplica más cercana por **redirección DNS** (un CNAME hacia el dominio de la CDN, que resuelve según la ubicación).
- HTTPS = HTTP sobre TLS (puerto 443). Lo ves en la guía 7.

5 · Tipos de ejercicio de la guía y cómo se encaran

A Conceptuales — ej. 1, 5, 8

- **Ej. 1**: 1 servidor, 1 IP y 1 puerto alcanzan (la 4-upla distingue las conexiones). Escalar es un tema de capacidad, no de puertos.
- **Ej. 5 (DNS ↔ HTTP)**: casi todo pedido HTTP a un host nuevo arranca con una consulta DNS, así que su latencia se suma.
  - *DNS centralizado*: cuello de botella, punto único de falla y servidor lejano para la mayoría → más latencia en **cada** HTTP.
  - *Sin caché*: cada pedido recorre raíz → TLD → autoritativo (varios RTTs extra) y la raíz se satura.
  - *Orientado a conexión*: un handshake extra por consulta (1 RTT más). Para 1 pregunta y 1 respuesta, UDP alcanza.
  - Con SMTP vale el mismo razonamiento, porque el saliente necesita MX + A. Pero el mail es asincrónico y tolera demora, así que el impacto **percibido** es mucho menor.
- **Ej. 8 (MIME)**: se agregan headers (MIME-Version, Content-Type, Content-Transfer-Encoding) y todo se codifica a ASCII (base64). SMTP y los servidores transportan texto sin interpretarlo, así que no cambian. Solo los **user agents** tienen que entender MIME.

B dig y registros — ej. 2

`dig nombre TIPO` muestra `nombre TTL IN TIPO valor`. Con `+trace` resuelve paso a paso desde la raíz (raíz → ar → uba.ar → dc.uba.ar). **Autoritativos** = los que figuran en los registros NS de la zona. **Primario** = el primer nombre del SOA; el resto de los NS son **secundarios**. La raíz y `ar` no tienen A ni MX: solo delegan.

C Escribir requests — ej. 3 y 9

```
GET / HTTP/1.1                  HEAD /tdc HTTP/1.1              GET /logo.jpg HTTP/1.1
Host: www.dc.uba.ar             Host: www.dc.uba.ar             Host: www.dc.uba.ar
                                                                If-Modified-Since: Mon, 05 Oct
2026 12:00:00 GMT
```

Ej. 9 (conferencia): el **POST** lleva el contenido en el cuerpo, con `Content-Type: application/pdf` (o el que corresponda, o `multipart/form-data`) y `Content-Length`, y el servidor responde `201 Created` + `Location: /recursos/42`. El **GET** de `/recursos/42` responde 200 con ese Content-Type y el cuerpo. El **HEAD** responde 200 con los mismos headers (Content-Type = el tipo pedido) y **sin cuerpo**.

D Contar RTTs de una página — ej. 6

1. Listá los objetos que se bajan **automáticamente** (HTML base + img, css, script, input image, iframe). Descartá <a> y <form>.
2. Aplicá la tabla de la sección 4: **1.0** = 2 RTT por objeto; **1.1 sin pipelining** = 2 + 1 por objeto; **1.1 con pipelining** = 2 + 1.
3. Escribí los supuestos: secuencial o paralelo, DNS ya resuelto o no (si no, +1 RTT al resolver por cada host nuevo), y que cada archivo entra en 1 RTT (el enunciado lo dice).

E Secuencia DNS con caché — ej. 7 (y la parte DNS del 4, 10, 11)

Para cada consulta anotá: **quién pregunta → a quién**, si es **recursiva o iterativa**, qué responde (**aa** o referral con NS + glue) y **qué queda en caché**. Reglas:

- El cliente le pregunta **recursivamente** al resolver, y el resolver pregunta **iterativamente** empezando por el ancestro más cercano que tenga en caché (si no tiene nada, por la raíz).
- Mail para un dominio → **MX** del dominio y después el **A** del servidor MX (si no vino en ADDITIONAL).
- Un **CNAME** obliga a seguir resolviendo el nombre canónico (puede caer en otra zona → más consultas).

- Se cachea todo lo recibido: answer, NS de AUTHORITY y glue de ADDITIONAL. Una segunda consulta a la misma zona va directo a su autoritativo.

Ej. 7: con `servidores.uba.ar` ya en caché, la consulta `uba.ar MX?` va directo y vuelve aa: MX `mailserver.uba.ar` + su A (208.25.19.2) en ADDITIONAL. Para `milagros.dc.uba.ar` el resolver le pregunta a `servidores.uba.ar`, que responde con un **referral** (NS `servidores.dc.uba.ar` + glue A 208.190.1.4); después le pregunta a ese y obtiene aa: A 208.190.1.15.

F Integradores — ej. 4, 10, 11, 12 (los más "de parcial")

Recorré la historia y, para **cada host nuevo que aparece**, aplicá la cadena **DNS → TCP → protocolo de aplicación**. El checklist:

1. **Actores**: PC/UA, resolver, proxy, servidor web, servidor de webmail, SMTP saliente, MX destino, servidor POP3/IMAP, servidores de terceros (CDN, ads, trackers).
2. **DNS** (UDP, resolver): un **A** por cada hostname distinto. **MX + A** para cada dominio de destino de mail (lo pide el **servidor saliente**, no la PC). Seguí los **CNAME**. Con caché vacía, recorré la jerarquía. Si los registros están en caché, no hay consulta.
3. **TCP**: una conexión por par cliente-servidor (en 1.0, una por objeto). Contalas cuando lo pidan.
4. **Aplicación**:
   - HTTP: un GET por el HTML y uno por cada objeto embebido; cada `<iframe>` es otra página con su propio GET.
   - SMTP: UA → saliente y saliente → MX, con la sesión HELO…QUIT.
   - POP3/IMAP para leer.
   - Webmail: el navegador habla HTTP y el servidor habla SMTP.
5. **Proxy**: la PC le hace todo el HTTP al proxy (y solo resuelve el proxy). Es el proxy el que resuelve y se conecta a los servidores de origen (si hay miss).

- **Ej. 4 (UDP)**: usan UDP solo la PC ↔ resolver y el resolver ↔ los name servers (DNS). Todo lo demás (HTTP, SMTP) va por TCP. Las conexiones TCP son: PC ↔ www.smn.gov.ar (una o varias, según la versión de HTTP y los objetos), UA ↔ SMTP saliente, y saliente ↔ MX de dc.uba.ar (esta última la desencadena el usuario, pero no la abre su PC).
- **Ej. 10 (spam)**: las consultas DNS son el A del SMTP saliente (en caché) y después, **por cada dominio distinto** de destinatario, su MX + A: uba.ar, dc.uba.ar, dm.uba.ar, df.uba.ar. Ojo: bob@uba.ar y alice@uba.ar comparten el MX, así que va una sola consulta. Para **mostrar** el mail, el UA del receptor hace GET de `ofertas.png` (www…) y del **tracker** `count.php?CampaignID=…&Email=…` (la imagen de 10×10, que **delata que abriste el mail**). El link de unsubscribe **no** se pide hasta que hacés clic.
- **Ej. 11 (jmail)**: `www.jmail.com` → A; `cdn.akamai.com` → **CNAME** a `recursos.jmail.com` → A (hay que seguirlo); el `<iframe src="http://ads.spameitor.com">` es otro dominio, así que lleva su DNS, su conexión y su página. El botón y el texto no generan pedidos. Contá las conexiones TCP por servidor distinto.
- **Ej. 12**: PC ↔ proxy (**HTTPS**, con CONNECT/túnel), proxy ↔ webmail (HTTPS), webmail ↔ SMTP saliente (SMTP), SMTP saliente ↔ servidor de mail destino (SMTP), PC destino ↔ servidor de mail (**POP3**). **El MX** del dominio destino lo consulta el **servidor SMTP saliente** cuando va a reenviar el mail, no la PC ni el webmail.

> **Para escribir en el parcial:** numerá los mensajes y aclará **desde qué dispositivo** sale cada uno (lo piden explícitamente: "aclarar de qué dispositivo se envía cada mensaje"). Indicá recursiva/iterativa y autoritativa/no. Escribí los supuestos: qué hay en caché, HTTP 1.0 o 1.1, conexiones paralelas o no.